SCURRY

Game design, systems research, and browser-development blueprint

Working genre: 2D top-down action RPG / life simulation / town-building sandbox
Target: Single-player PC browser game, playable from GitHub Pages
Proposed technology: Phaser 4 + TypeScript + Vite, with locally stored saves
Report status: Design proposal; all Scurry costs, stats and formulas are playtest hypotheses, not finalized game values
Research date: October 12, 2026

> **Pitch:** You are a small earthworm with an ordinary day planned: dig a little, earn a little, go home. One unexpected discovery beneath the town of Clovercross gives you a choice. Help your neighbours rebuild a struggling woodland settlement, turn its resources into a private fortune, or play both sides. Farm, dig, fight, trade, build, and live with the consequences as your burrow becomes a home, a business, and potentially the heart of a growing town.

---

1. Executive design decision

Scurry should be an action RPG first, with a persistent settlement simulation underneath. The player always controls one character directly, rather than becoming an omnipotent city-planning cursor. Town improvements should result from deeds, construction contracts, land rights, shared projects, businesses, and alliances. This preserves the intimacy of Pokémon and Harvest Moon while borrowing the meaningful resource constraints and consequences of SimCity, Age of Empires, The Sims, and The Oregon Trail.

Its defining promise is **the town remembers**. A repaired bridge brings customers to the market. A stolen pump causes shortages. A well-defended village attracts settlers. Cutting down too much forest increases landslide and fire vulnerability. A friendly neighbour may help defend your farm; a betrayed neighbour may refuse to sell you seed. These are *mechanical* consequences, not dialogue-only morality labels.

Five pillars

1. **Live small, affect big.** Every crop, tunnel, business, friendship, theft, and repair can influence a larger settlement.
2. **Be a citizen—or an opportunist.** Legal, generous, selfish, deceptive, and criminal routes are viable, but not consequence-free.
3. **Work the land; explore what lies below.** The worm's dig ability is traversal, gathering, puzzle-solving, building, and combat utility.

4. **Fight to protect or to profit.** Crisp real-time hack-and-slash combat, equipment, enemies, raids, health, and rescue/recovery.
5. **Tell systemic stories.** Seasons, weather, trade, NPC needs, faction reactions, and disasters combine to make each save different.

Important scope boundary

Do **not** reproduce all six games in full. The first release should have **one small town, one farm, one underground area, one nearby wilderness map, roughly 12–16 named townsfolk, six crop types, six buildable structures, and three enemy archetypes**. A functioning, interconnected small simulation will be much more satisfying than six shallow standalone systems.

---

2. Research: what to borrow from the six classics

These are design interpretations, **not claims that Scurry will duplicate original code or assets**.

Source game	Historical gameplay systems	Most valuable Scurry translation	Avoid copying
The Oregon Trail (especially MECC's 1985 Apple II edition)	Finite supplies, daily pace/ration decisions, weather-driven risk, injuries/illness, trade, branching travel, river crossings and unpredictable setbacks	Overland **merchant caravans**, supply preparation, travel events, flooding, disease/pest outbreaks, repair decisions and text-led expedition encounters	A literal historic trail, wagon UI or the game's exact writing/art
Harvest Moon (1996 SNES)	A neglected farm, clearing fields, tools/stamina, seasonal crops, watering, shipping produce, animals, relationships, festivals, a limited day	Core farming, farm expansion, tools, weather, four seasons, market shipping, gifting, festivals and a restorative daily routine	Crop sprites, characters, map layouts and exact timing/economy
Age of Empires (1997; reference also to its successors)	Labor gathers resources; wood/food/stone/gold feed construction, upgrades, technologies, population expansion and defenses; raids disrupt economic work	Resource chains, workshops, gathering contracts, town projects, defensive upgrades, production specializations and faction-controlled supply routes	Large RTS army command, 100s of constantly simulated units
SimCity (1989)	Zone/access/service dependencies, budget/tax income, crime, pollution, happiness and disasters; growth emerges from infrastructure	Connected paths, housing capacity, water/food/safety coverage, treasury, upkeep, growth bottlenecks and disaster resilience	Micromanaged power grids or full city zoning in the first release
The Sims (2000)	Needs and mood influence behaviour; homes, jobs, routines, relationships, objects and careers interact	NPC hunger/rest/safety/income/social needs, schedules, friendships, work attendance, trading preferences and home comfort	Eight high-maintenance needs for the playable worm
Pokémon Red/Blue (Game Boy)	Readable grid-based overworld, memorable towns/routes, NPC dialogues, environmental blockers, compact interiors and incremental unlocking	Legible 16-pixel tile language, town/route/dungeon structure, short interactions, exploration unlocks and a portable-game feel	Pokémon creatures, battle system, game maps, dialogue, sprites or iconography

Key distinction: Pokémon Red/Blue's battles were turn-based. **Scurry's combat is original real-time top-down hack-and-slash.** Pokémon is a reference for movement, art readability and exploration structure, not for its combat implementation.

The Oregon Trail's strongest contribution is not simply 'random bad things happen.' Its decisions deliberately trade safety against time and resources; the 1985 game's designer describes how that edition added many now-recognizable systems. Translate this into *forecast → prepare → choose risk → experience outcome → recover*.

---

3. Setting, tone and opening

3.1 The world

Set Scurry in **Underbough Valley**, where birds, beetles, cats, dogs, worms, moths, mice and other anthropomorphic creatures run a compact, ramshackle community at garden scale. A bottle cap might become a shop sign; a cracked teacup becomes a public fountain; roots are bridges, and a misplaced boot can block a mine passage.

Starter town: Clovercross, population approximately 14, with a handful of homes, a general store, bakery, town hall, little market square, repair shed, seed stand and watch post. The player lives just outside town in a neglected burrow with a tiny garden. Spaces should feel connected and spatially consistent: doors align with entry tiles, interiors match exteriors, entrances are discoverable rather than arbitrary invisible triggers.

Tone: Warm and funny on the surface, with credible social tension beneath. NPCs have mixed motives. A scavenger cat is not automatically evil and a respectable bird councillor is not automatically trustworthy. Violence is non-graphic but consequential. Every species should be characterized individually, not morally stereotyped.

3.2 Opening playable sequence: 'Just Another Dig'

00:00–02:00 — Wake up. The player, default-named **Nim**, is a small earthworm. The first screen is Nim's simple burrow. The player sees a note: 'Day off. A little digging. Maybe a nice lunch.' Learn movement, inspect a cupboard, and step outside.

02:00–04:00 — Dig the garden. A neighbour asks Nim to clear three clods of soil. Learn the directional dig action, stamina, picking up clay/pebbles and planting one seed. An old pipe and strange carved root are found below the plot.

04:00–07:00 — First combat. A pair of invasive thorn grubs emerge from the disturbed earth. Learn one melee swing, facing, damage, knockback and recovery food. Make the encounter easy and forgiving; defeat is impossible to make permanently costly during this tutorial.

07:00–10:00 — First civic choice. The buried object appears to be a component from the town's aging irrigation mechanism. The player can **return it to the public works beetle**, **sell it to the night-market trader**, or **keep it for private use**. Choices update separate state variables rather than a single good/evil label.

10:00–15:00 — Town feedback. The player enters Clovercross, meets two named residents and receives a visible effect: a restored community water source; cash but reduced trust and worsening crops; or a private shortcut/upgrade at the cost of delay. The first day ends with a newspaper-style **Town Bulletin** showing consequences.

Agency safeguard: The main story proceeds through all three choices. Never permanently soft-lock the player because they chose an unapproved moral option. The community may recover by other means, at a cost.

3.3 Long-term story

The discovery hints at an old network of subterranean waterways. Various factions want control because it could make Clovercross prosperous, flood its streets, or open smuggling routes. The player chooses where, why and for whom to dig. Three high-level ending states are possible, but the sandbox continues: **Community Haven**, **Trading Powerhouse**, or **Shadow Borough**. Mixed outcomes are also legitimate.

---

4. World layout and traversal

World structure for version 1

Map	Suggested logical dimensions	Purpose
Clovercross hub	96 × 96 tiles	Homes, shops, services, social life, town works and raids
Nim's Burrow & Farm	48 × 48 tiles	Home, farming, storage, construction, personal economy
Root Tunnels	64 × 48 tiles	Diggable walls, minerals, puzzle shortcuts, enemies
Bramblewood Edge	64 × 64 tiles	Wood/fiber, foraging, hostile encounters, trading road

All dimensions are **design targets**, not a requirement to create 9,000 unique tiles. Reusable tilemaps make this practical. One more small wetland region can follow after the vertical slice.

Movement: Free directional movement within grid-aligned scenery, collision on solid tiles, four- or eight-direction sprite animation, camera follow. Interaction happens on the nearest valid tile in front of the character. Consistently place exterior door triggers directly in front of the drawn threshold; put the matching interior exit on the indoor doorway.

Earthworm-exclusive mechanics:

- **Dig:** Remove eligible loose-earth tiles and expose items, tunnels or hazards. Cannot dig stone, reinforced walls, permanent quest boundaries, or another resident's floor without permission.
- **Burrow Dash:** A short underground displacement that dodges damage and crosses specific soft-ground obstacles. Costs stamina; has a cooldown.
- **Soil Sense:** Brief pulse highlights buried roots, minerals, weak tunnel walls and water leaks within a radius.
- **Tunnel Building:** With materials and municipal permission (or deliberate illegal digging), connect approved entrances and shorten trade or rescue routes.
- **Terrain consequences:** Unreinforced tunnels may collapse during heavy rain or after excessive excavation; bracing and drainage are meaningful upgrades.

Require a **terrain metadata layer** (`soil`, `rock`, `water`, `protected`, `owned`, `diggable`, `hazard`) rather than treating visual pixels as collision truth.

---

5. Fundamental time, survival and progression

5.1 Clock and calendar

Proposal: 12 real-time minutes per normal in-game day, approximately 14 days per season, four seasons per year. Sleep advances to morning. Pause the day during long dialogue, menus and building placement;

provide an optional slower-day setting. Weather is sampled at dawn, updated at a few predetermined beats, and exposed in a forecast board.

Morning: NPCs wake, supplies are consumed, crops advance and town accounts reconcile. Day: exploration, construction, work, NPC business hours. Evening: closing shops and social events. Night: riskier paths, night markets and some intruders; no mandatory combat timer.

5.2 Player survival (lightweight, not a chore simulator)

Stat	Starting value	How it changes	Recovery
Health	100/100; drawn as five large hearts	Combat damage, falls, extreme events	Meals, herbal poultices, clinic, rest
Stamina	100/100	Digging, chopping, tilling, dodge, sprint	Short recovery, food, sleep
Fullness	80/100	Gradual daily decline; lowered by hard labor	Eating; low fullness weakens stamina recovery
Morale	60/100, generally kept in Journal	Friendships, shelter, rewarding activity, major losses	Visiting neighbours, hobbies, resting at home

NPCs can instead use a simpler need model: **food, rest, safety, work/income, social connection**. Each need affects scheduled behavior but does not require expensive continuous simulation. A resident whose shop was damaged may repair it or request help rather than opening on time.

5.3 Personal progression

Track **Digging, Farming, Crafting, Trading, Combat and Civic Influence** (levels 1–5). Skill advancement unlocks abilities or efficiency, not just stat inflation. Examples: Digging II reveals minerals; Farming III improves quality of fertilized crops; Trading II reveals price demand; Civic Influence III permits larger town proposals. Never gate the entire good or bad path behind a single skill.

---

6. Farming, harvesting and business

6.1 Farm interaction loop

Clear → till → fertilize (optional) → plant → water/irrigate → grow → harvest → eat, craft, sell, gift or donate. Dirt retains a watered flag for the day; growth progresses at dawn, not on every rendered frame. Rain waters exposed plots. Soil fertility, crop season, water access, pest pressure and construction runoff affect yields. Start with four crop types, growing to six in the full first release.

Example balancing targets (illustrative; to tune in playtests):

Crop	Season	Growth	Seed cost	Typical sale	Special feature
Turnip	Spring	3 days	4 shells	12 shells	Quick early cash
Carrot	Spring/Autumn	4 days	6	17	Cooking and contracts
Berry	Summer	5 days initially	12	9 per pick	Regrows every 2 days
Mushroom	Autumn	4 days	7	19	Can grow in dark damp plots
Winterroot	Winter	6 days	10	29	Resistant to frost
Cabbage	Spring/Autumn	5 days	8	22	Feeds several residents

Yield quality and market demand modify the actual sale price. Do not rely on endless vendor gold: merchants have daily demand and inventory rules, but always provide a baseline public shipping bin so the player cannot become unable to earn money.

6.2 Economy and currency

Currency: **shells** (ş in UI if the font supports it; otherwise use a shell icon). Resources: **wood, fiber, stone, clay, metal scraps, crops/food, and water access**. Water is an infrastructure capacity/resource indicator, not necessarily a physically carried item.

There are **two distinct accounts**:

1. **Personal inventory / wallet:** Nim's shells, tools, seeds, crafted materials and owned buildings.
2. **Town ledger / treasury:** Community materials, services, maintenance expenses, public projects and tax/trade income.

A public project should not silently consume the player's entire personal inventory. The player can *donate* materials, *take a paid contract*, or *propose a treasury-funded project*. Each has different reputation and financial effects.

Local economy model

At dawn, evaluate production and consumption once, not every animation frame:

```
netFood = farmOutput + NPCFarmOutput + importedFood - residentConsumption - spoilage

townTreasuryNext = treasury + businessTax + tradeFees + grants - upkeep - emergencyCosts - projectPayments

shopPrice = clamp(basePrice × (1 + demandPressure) × seasonFactor × factionModifier, minPrice, maxPrice)
```

Limit demand pressure to a small, readable range (for example ±25%). Apply **bounded, gradual price changes** to avoid runaway inflation or an unwinnable starving economy. Baseline NPC farms and imports already feed most residents; Nim's farm makes a *difference* but does not single-handedly support a whole town.

Business ownership

Begin with **farm shipping and small repair contracts**. Unlock one personally owned **roadside produce stall**, later a **workshop**, and eventually leases or partnerships with town vendors. Business revenue is calculated from stock availability, foot traffic, local reputation, prices, and upkeep. A shady business might turn a higher short-term profit while increasing watch patrols, heat and the likelihood that legitimate suppliers withdraw.

---

7. Construction, town simulation and citizen decisions

7.1 Building without turning Scurry into a full RTS

Use two modes:

- **Personal placement:** Within owned farm/burrow parcels, the player directly positions fences, beds, paths, storage, irrigation and workstations in snapped tile footprints.
- **Town projects:** Within the municipality, the player proposes or purchases permits for upgrades such as a bridge, market, water tower, repair shed or watch post. Building requires footprint clearance, material delivery, labour and a timed construction phase.

NPC artisans work visibly on a project; the underlying progress is a deterministic daily calculation. Buildings have `footprint`, `entrance`, `type`, `owner`, `health`, `level`, `upkeep`, `requirements`, `effects` and `repairCost` metadata. A placed structure updates collision and pathfinding *immediately*.

SimCity-lite town meters: Population, **housing**, food supply, water service, path connectivity, safety, commerce, ecology and overall satisfaction. Map overlays should indicate service gaps. Growth requires sufficient combined services, not a single arbitrary XP threshold.

Infrastructure rule examples: A new home needs walkable path access, adequate drinking water and vacant housing capacity; a business requires a reachable customer path and stock; a watch post increases nearby safety but consumes treasury upkeep; a water cistern lowers drought risk within its service radius. Larger buildings cannot block every route to critical shops.

Starter construction catalogue

Structure	Primary cost categories	Concrete effect
Garden fence	Wood/fiber	Reduces crop trampling and pest chance
Burrow extension	Clay/wood/shells	Additional indoor storage, comfort, NPC visit space
Drainage ditch	Digging labour/stone	Redirects floodwater from certain tiles
Footbridge	Wood/stone/community labour	Restores route access; improves trader visits
Market stall	Wood/cloth-like fiber/shells	Unlocks player retail income
Watch post	Wood/stone/treasury upkeep	Reduces raid damage in coverage area

Later expansions: clinic, fire bell, public greenhouse, school/library, workshop and caravan depot.

7.2 Morality with multiple dimensions

Do not use a single 'good vs. evil' slider. Track:

- **Community Trust** (0–100): Will townsfolk rely on Nim?
- **Heat / Suspicion** (0–100): Are guards and merchants suspicious of crimes?
- **Faction Standing** (–100 to +100 each): How does each group feel about Nim?
- **Ecology** (0–100): Long-term soil, water, trees and wildlife health.
- **Town Prosperity and Safety** (separate measurable systems): Can residents thrive and feel protected?
- **Individual NPC relationships** (–100 to +100): Friendship, resentment and loyalty.

Witnesses matter. A theft is not instantly known to every resident. Witnessed actions create an evidence event; town gossip can carry it through social links, with an occasional delayed discovery. Restitution, apology, community service and legal penalties allow recovery. Organized sabotage yields stronger immediate rewards but more severe investigations. Avoid making 'bad' gameplay simply an option that disables half the game.

Example outcomes (starting balance proposals):

Action	Immediate payoff	Delayed systemic effect
Repair the public pump	Materials and time spent; small contract fee	Better water coverage, higher trust, healthier crops
Sell the missing pump component secretly	+60 shells	Trade-guild favour, +heat if exposed, irrigation risk

Action	Immediate payoff	Delayed systemic effect
Keep the component for private irrigation	Better farm yield	Neighbours have slower repairs and some relationships worsen if discovered
Replant harvested timber	Time and small expense	Improved ecology and lower erosion/flood risk
Steal from a trader	Goods without cost	Witness investigation, merchant refusal, bounty/ restitution quest
Defend town during raid	Combat and medical risk	Safer streets, higher trust, new guard contracts

Four early factions: Clovercross Council (services and permits), **Rootworkers Union** (diggers/builders), **Mosskeepers** (soil, forests, waterways), **Lantern Market** (traders, legitimate and clandestine). **Thornband** is an external aggressor network rather than proof that any particular animal species is 'bad'. Different named characters can disagree even within the same faction.

8. Combat, danger, health and intruders

8.1 Controls and fighting feel

Top-down, real-time, direction-based melee. Nim's starting weapon is a small gardening trowel that serves as a tool and defensive weapon. Facing determines the hit arc. A visible attack wind-up, hit frame, knockback and brief post-hit invulnerability make combat understandable.

Recommended first combat parameters:

- Basic swing: 8 damage; approximately 0.38 sec between attacks; 85-degree forward arc, 1.3 tiles reach.
- Heavy swing (later): 15 damage, costs 15 stamina and leaves an opening.
- Burrow Dash: 20 stamina, brief invulnerability only during a short designated window, 2 sec cooldown.
- Invulnerability on taking damage: 0.6 sec, with flashing outline.
- First grub enemy: 24 HP, 5 contact damage, telegraphed lunge and clear attack pause.
- Enemy pursuit: only within a specified territory and line-of-sight / hearing range; give up after a cooldown.

These are **initial tuning values**, not historical facts. Tune by measuring average time-to-defeat and player damage taken. Avoid stun locks, invisible attacks, walls that trap the player and an enemy spawn on top of a door trigger.

Damage model

```
finalDamage = max(1, round(baseDamage × attackModifier × resistanceModifier - flatArmour))
```

Use a short invulnerability timer and non-overlapping damage windows. Friendly townsfolk are not automatically valid targets unless a purposeful hostile-crime action is supported with clear confirmation and predictable legal consequences. No gore is needed.

Defeat: Nim collapses and wakes at home or clinic the next morning with restored partial HP, some lost time, a modest medical fee or recoverable dropped pack. No arbitrary irreversible death for the introductory game. Build and town damage persist, so defending matters.

8.2 Enemies and protective NPCs

- **Thorn grubs:** Basic close-range attack; common in disturbed soil.

- **Brambler beetles:** Armoured charging intruders in the woods; vulnerable from side or after a missed charge.
- **Thornband scouts:** Intelligent raiders with patrols, retreat and theft of stored materials; mixed animal species.

Guards, neighbours and allied faction members can react with lightweight state machines: idle → investigate → warn → defend/flee → recover. NPCs should not rely on the player for all defense, but investment in roads, guard posts and public goodwill improves how effectively they respond.

8.3 Events and disasters: the Oregon Trail connection

Event families: **heavy rain/flood**, **windstorm**, **drought**, **forest fire**, **tunnel collapse**, **crop pest outbreak**, **caravan breakdown**, **market shortage**, and **hostile raid**. Not every event is a combat event. Use conditions and warning signs:

```
eventRisk = baseSeasonRisk × weatherFactor × exposureFactor × (1 - mitigationFraction)
```

A flood is much more likely in low wetlands during sustained rain and less costly with functioning drainage. A fire is more damaging with accumulated brush and without a watch/fire bell. Raid likelihood reflects road prosperity, patrol strength, faction tension and whether a protected supply store exists. Do not let every clock tick independently roll disasters.

Event state machine: `forecast → warning → active → response → recovery → cooldown`. A starter town should see at most a minor early-game event while the player learns. Escalate only after defenses, town scale and player competency increase. Disasters must offer meaningful countermeasures and recovery quests instead of deleting hours of player work without warning.

---

9. Trade journeys inspired by The Oregon Trail

A crucial fusion mechanic should arrive *after* core town play works: **caravan dispatches**. Nim can personally escort a merchant, appoint a trusted NPC, or buy passage. Each trip crosses two or three short regions or event nodes with finite food, repair items, cargo space and travel time.

Before leaving, choose **route** (safe/slow vs shortcut/risky), **supplies**, **guard escort**, **pace** and **cargo**. During the journey, a compact illustrated bulletin or in-world encounter offers choices: repair a bridge, ford a creek, negotiate a toll, assist a stranded traveller or raid the convoy. Unlike Oregon Trail's wagon era, these are animal-scale trade journeys under hedges and across streams.

Consequences reach the town economy. A successful run brings seeds and metal scrap; a robbery causes shortages; saving a rival merchant builds faction standing; smuggling can bring exceptional private profit and law-enforcement heat. **The trip must not be a completely separate game**: cargo changes storefronts, construction progress and personal relationships back home.

Version-1 scope: One route, two possible encounters and one destination merchant. Expand later to a branching network and repeatable procedural dispatches.

---

10. NPC simulation, memory and quests

10.1 Named cast (examples)

Resident	Species / work	Need or ambition	Potential storyline
Nim	Earthworm / player	Find purpose beyond small jobs	Build, exploit or transform Clovercross
Bram	Beetle / public works	Repair broken irrigation	Can become a mentor or adversary
Tilly	Mouse / baker	Dependable grain and produce	Supply contracts, festival treats, theft investigation
Rook	Crow / trader	Reliable goods and influence	Legitimate caravan links or gray-market deals
Mara	Cat / night-market vendor	Keep her struggling family business open	Not simply a criminal character
Gable	Dog / watch captain	Keep the town safe without abusing residents	Evidence, warrants, patrols, community defense
Fern	Beetle / ecologist	Restore a polluted stream	Planting, drainage and conservation rewards
Lumi	Moth / medic	Enough medicine for local needs	Herb cultivation and clinic improvements

Populate the rest of town with residents who have jobs, small personalities, home locations, schedules and relationship networks. **Persistent NPC identity is more valuable than a very large generic crowd.**

10.2 Schedule and needs architecture

Use a schedule template per NPC: wake, travel, job, meal, leisure, home, sleep. An event can interrupt it. At active range, run simple path-following and reaction AI; offscreen, advance a lower-frequency *abstract schedule state* without simulating every step. Only load detailed sprites for active maps.

NPC memory events might include `HELPED_WITH_REPAIR`, `STOLE_WITNESSED`, `SOLD_GOOD_CROPS`, `DID_NOT_DEFEND_RAID`, `RETURNED_MISSING_TOOL`, and `GIFTED_ITEM`. Use a bounded list with timestamps, expiry and personal salience. Quests subscribe to these outcomes through named triggers instead of scanning dialogue text.

Quest framework

A quest is structured data with: `id`, `giver`, `prerequisites`, `objectives`, `completionRules`, `rewards`, `consequences`, `dialogueKeys`, and `failOrAlternativePaths`. Objectives may listen for `harvest`, `deliver`, `talk`, `defeat`, `build`, `repair`, `explore`, or `choose` events. Design quests with at least two possible outcomes when they involve community resources.

---

11. The coupled simulation: one worked example

Event: 'The Thirsty Square.' Heavy rain has washed out the west bridge and a cracked public pump is failing.

Public-benefit route: Nim mines stone and gathers wood, completes a paid bridge repair, and donates two harvested crops to the communal kitchen. Connectivity and water service rise; trade foot traffic improves; the baker sells more food; residents are happier; defence patrols can reach the western district faster.

Profit-first route: Nim privately repairs a shortcut tunnel to their own stall and buys up the last water jars. Personal sales increase, but town trust and satisfaction fall as residents pay more; the watch asks questions if a merchant reports hoarding.

Criminal route: Nim steals pump parts for the Lantern Market. The night market pays well, but if witnesses connect the theft to Nim, a warrant, fines, damaged relationships and possibly an embargo follow. Nim can evade, pay restitution, expose another corrupt party or repair the damage.

Environmental route: Nim chooses a drainage ditch rather than paving the wetland. It is slower to build but reduces future flood exposure and helps the Mosskeepers. A cheaper shortcut would have increased road traffic but raised later flood risk.

All outcomes should update **one underlying simulation**. The HUD, Bulletin, NPC conversations and map overlays merely reveal different parts of that shared state.

Rules for systemic integrity

- Every item comes from a known source and goes to a known sink.
- NPC sales should decrement stock and increase revenue; money cannot appear twice.
- Ownership and crime use the same interaction hooks as normal item pickup.
- Bridges/doors/buildings update navigation blockers atomically.
- Crop growth happens once per day, including when the player sleeps multiple days.
- A structure destroyed during a raid immediately loses its service benefits.
- Events publish journal entries with cause, effect, witnesses and recovery options.

---

12. Retro art and interface direction

12.1 Original visual style

Mix **Game Boy-era readability**, **SNES-era warm farm detail**, and **early home-computer event presentation** without lifting any copyrighted graphics.

- **Camera:** Fixed top-down three-quarter character sprites, with 2D tiled environments, restrained elevation effects and depth sorting for trees/doorframes.
- **Logical pixel resolution: 512 × 288**, ideally displayed in integer scale multiples where window size permits. Use nearest-neighbour filtering and crisp pixel alignment.
- **Tiles:** 16 × 16 pixels. Buildings commonly 2 × 3 up to 7 × 6 tiles; directional animated creatures approximately 16 × 24 to 24 × 32 pixels. Keep collision geometry smaller than oversized decorative hats, tails or leaves.
- **Palette:** Earthy forest green, worn gold, clay orange, cream, deep moss shadow, muted blue and warm lamps. Cap individual sprite palettes before adding global lighting accents.
- **Creature designs:** Readable species silhouette at 1×, expressive eyes and 3–6 frame action animations. Animals wear practical handmade clothes, bags and tools.
- **World art:** Tiny improvised architecture from wood scraps, bark, pebbles and repurposed human objects. Avoid photorealistic textures.
- **Oregon Trail flavour:** Monospace-style 'Town Bulletin', weather and expedition cards; restrained black/cream/green linework, tiny icons, short event prose; no recreation of the original UI.
- **Sound:** Light ambient forest beds, percussive soil-dig ticks, 8/16-bit-inspired melodic motifs, soft footfalls and highly readable combat cues. Respect mute/volume settings.

Research image references (not for reuse as assets)

The early Pokémon Game Boy town screenshot illustrates tiny readable spaces and tile discipline; original Harvest Moon screenshots show warm farming detail; Oregon Trail screens show concise, legible status/event framing. They are reference material only; create **new original designs** for Scurry.

12.2 Proposed PC HUD

Top-left: Five health hearts + stamina line + fullness icon. **Top-right:** Day, weather, time, shells. **Bottom-left contextual:** Current tool and action prompts. **Bottom-centre:** Four inventory quick slots. **Bottom-right:** Small minimap and quest arrow, optional. Menus: Inventory, Farm, Town Ledger, Relationships/ Factions, Journal, Map, Building, Settings.

Town Bulletin appears on sleep and major events: three clear summaries, such as `Bridge restored`, `+2 new visitors`, `Lantern Market suspects dishonesty`. Clicking a report opens the supporting cause without revealing secret investigation data the player has not learned.

Keyboard / mouse proposal

Input	Action
WASD / arrow keys	Move; diagonal normalized
E	Talk / use / interact; context-sensitive
Space or left click while equipped	Attack or use active tool; hold for repeated farming if accessible
Shift	Sprint (stamina cost)
K	Burrow Dash / dodge
1–4	Quick tool slots
I	Inventory
B	Build / placement mode when in eligible area
M	World map
J	Journal / quests
Tab	Town ledger and reputation
Esc	Pause/settings; release pointer

Make controls remappable in settings and avoid hard-coding game logic to specific keys. Mouse UI should support windows and tooltips; gamepad can follow later. Include full-screen and windowed support, scalable text, screen-shake toggle and a high-contrast indicator for interactive objects.

13. Technical plan for a free static browser game

13.1 Recommended stack

Component	Recommendation	Reason
2D game engine	**Phaser 4.2.1** (pin exact version once tested)	Browser-native 2D scenes, sprites, animation, tilemaps, cameras and Arcade physics; open source
Language	**TypeScript**	Prevent common state, event and content-data errors as the simulation grows
Bundler	**Vite**	Fast local development and static production build
Tile maps	**Tiled** JSON export	Visual editor, tile layers, collision and object layers
Art tools	LibreSprite / Krita / Aseprite if already owned	Pixel sprites and atlas editing; choose free options first
Save storage	**IndexedDB** with autosave and export/import JSON	Supports offline single-player persistence without paid backend
Testing	Vitest plus Playwright smoke test when ready	Validate economy/time/event logic and deployment
Hosting	**GitHub Pages**	Static HTTPS site; no server bills for single-player gameplay

Phaser 4 is a **2D, WebGL-based engine**, not a complete RPG maker. A browser with compatible WebGL support is required. Its framework provides rendering, input and physics primitives; **it does not create farming, economic simulation, NPC memory, combat design or quests automatically**. Keep those original systems in independent TypeScript modules.

13.2 Authoritative game-state architecture

Store all persistent facts in one serializable world state, while treating sprites, sound instances, UI objects and runtime physics bodies as transient render state.

```
type ScurrySave = {
  schemaVersion: 1;
  seed: number;
  day: number;
  minuteOfDay: number;
  season: 'spring' | 'summer' | 'autumn' | 'winter';
  player: { mapId: string; x: number; y: number; hp: number;
            stamina: number; fullness: number; wallet: number;
            skills: Record<string, number>; inventory: ItemStack[] };
  maps: Record<string, { changes: TileChange[]; objects: SavedObject[] }>;
  crops: CropState[];
  buildings: BuildingState[];
  npcs: Record<string, NpcState>;
  factions: Record<string, number>;
  town: TownSimulationState;
  quests: Record<string, QuestProgress>;
  events: WorldEventRecord[];
};
```

Single authority: `GameState` owns factual values; `EconomySystem`, `FarmingSystem`, `QuestSystem`, `FactionSystem`, `BuildingSystem`, `CombatSystem`, `NPCSystem`, `WeatherSystem` and `EventDirector` propose transactions. A typed

EventBus broadcasts successful outcomes. Each action validates prerequisites, applies changes once, and then publishes a result. A failed purchase leaves all balances unchanged.

Simulation timing: Phaser renders around 60 frames/second; movement/combat update every frame using delta time or a fixed combat time step; NPC decisions maybe 4–10 times/second nearby; town economy, population, farming and services update **once at dawn** or when a world-changing event requires recalculation. Do not run full city logic 60 times/second.

Pathfinding: Small-grid A* for nearby NPCs, recalculated on obstruction changes; cache region connectivity for town macro calculation. Use a **finite set of entrance nodes** rather than unrestricted NPC building entrances.

13.3 Repository layout

```
scurry/
├─ .github/workflows/deploy.yml
├─ index.html
├─ package.json
├─ package-lock.json
├─ tsconfig.json
├─ vite.config.ts
├─ README.md
├─ LICENSE
├─ public/
│  └─ assets/
│     ├─ sprites/        # Original sheets and atlases
│     ├─ tilesets/       # Original 16px tiles
│     ├─ maps/           # Tiled exports
│     ├─ audio/
│     └─ fonts/
├─ src/
│  ├─ main.ts
│  ├─ game/config.ts
│  ├─ scenes/             # Boot, World, Interiors, UI, Pause
│  ├─ entities/           # Player, NPCs, enemies, projectiles
│  ├─ systems/            # Combat, farming, day cycle, town, etc.
│  ├─ simulation/         # State, event bus, transactions, services
│  ├─ data/               # NPC, crops, items, recipes, quests (JSON)
│  ├─ ui/                 # HUD, dialogue, journal, builds, ledger
│  ├─ save/               # IndexedDB, migrations, export/import
│  └─ util/
└─ tests/
   ├─ systems/
   └─ save/
```

Content-driven expansion

You can add **new crops, recipes, dialogue, buildings, NPC schedules, event text, shop stock and most quests** by editing JSON files **if they only use existing behaviours/objective types**. New gameplay verbs—such as a fishing minigame, a new combat mechanic or a genuinely new disaster simulator—**require TypeScript changes**. GitHub Actions rebuilds on commit; the player does not need to reinstall an application.

---

14. GitHub Pages deployment recipe

GitHub Pages can serve Scurry because the initial game is a **static web app**. Vite compiles TypeScript and bundles game files to `dist/`, and GitHub Actions publishes that folder. GitHub Pages itself does not run Node/TypeScript server code or keep authoritative multiplayer data.

14.1 Local PC development

1. Install an actively supported **Node.js LTS** release and Git for Windows (or use the GitHub Desktop client).
2. Create a Vite + TypeScript project, add Phaser, initialize Git, and commit it.
3. Run `npm install` and then `npm run dev` in the project directory.
4. Open the localhost URL printed by Vite. Use `npm run build` and `npm run preview` before publishing.
5. Use the browser developer tools to monitor exceptions, network asset failures and save migrations.

Typical first commands (after creating a `scurry` repository):

```
npm create vite@latest scurry -- --template vanilla-ts
cd scurry
npm install
npm install phaser@4.2.1
npm run dev
```

The generated TypeScript template must then be replaced with the actual Scurry game architecture; these commands install a scaffold, **not a playable RPG**.

14.2 Vite base path

For a repository named `scurry`, GitHub Pages normally hosts at `https://YOURUSERNAME.github.io/scurry/`; set the Vite base accordingly. For a custom domain / username root site, use `/` instead.

```
// vite.config.ts
import { defineConfig } from 'vite';

export default defineConfig({
  base: '/scurry/',
});
```

Assets loaded by runtime URL strings must also respect `import.meta.env.BASE_URL` or use Vite asset imports. An absolute `/assets/...` path will often break on repository Pages. For example:

```
const tileUrl = `${import.meta.env.BASE_URL}assets/tilesets/town.png`;
```

14.3 Automatic deploy workflow

Create `.github/workflows/deploy.yml`:

```
name: Deploy Scurry to GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
```

```
  pages: write
  id-token: write

concurrency:
  group: github-pages
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: npm
      - run: npm ci
      - run: npm run build
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v4
        with:
          path: dist
      - id: deployment
        uses: actions/deploy-pages@v4
```

Then: **GitHub repository → Settings → Pages → Build and deployment → Source: GitHub Actions**. Push to `main`, view the Actions status, and open the Pages URL. GitHub versions of official Actions should be checked against their current documentation when actually installing the workflow.

14.4 Save, privacy and offline behaviour

- Save versioned state with IndexedDB, including crops, modified map tiles, buildings, NPC relations, completed quests, faction values and deterministic RNG seed.
- Autosave at **sleep, map transition and major project completion**; keep an alternate previous valid save so one interrupted write cannot destroy a campaign.
- Support explicit **Export Save** and **Import Save** JSON buttons, since browser saves are tied to a browser profile and website origin, not the user's GitHub account.
- A Pages game **does not automatically sync saves between PCs**. Optional cloud saves would require a separately specified backend/connector, outside the free static MVP.
- Avoid silently caching corrupt state forever. Use schema migrations and user-visible backup/recovery notifications.
- Future progressive-web-app offline caching can be added after the regular GitHub Pages build works reliably.

---

15. Build roadmap and acceptance gates

Stage	Target	What must be playable / verifiable	Not yet included
0. Visual/technical prototype	Basic feasibility	One pixel map, controlled worm movement, collision, correctly aligned building doors/interiors, camera, one NPC, save/reload	Economy or complex town
1. Core vertical slice	**First 15-minute Scurry experience**	Dig, plant, grow, harvest, simple melee, take damage/heal, talk to 3 NPCs, make first civic choice, persist consequences, browser publish	20+ buildings, advanced factions
2. Living town	Connected simulation	Six to ten active NPCs on schedules, player wallet and town ledger, one market stall, 3 buildings, bridge connectivity, town service overlay	Full seasons or many regions
3. Consequences & conflict	Meaningful choice	Faction/individual reputation, witnessing/heat, guards, one raid, one flood, preventive building, clear recovery routes	Extensive procedural events
4. Replayable early-access sandbox	Sustainable play	Four seasons, six crops, 12–16 NPCs, 6 build types, 3 enemy types, 1 caravan route, questlines, town bulletin, balancing and full save import/export	Giant map or multiplayer
5. Extended game	Depth and polish	New districts, deeper underground, player businesses, public votes, more ecology/disasters, NPC households, story endings	Only after player testing

Gate before adding a new system: It must affect at least **two other existing systems** and produce a visible player-facing consequence. Example: the bridge must influence both route access and commerce; the flood must influence both physical tiles and the town ledger. This prevents disconnected feature bloat.

Reasonable effort expectation: For one developer with off-the-shelf tools but **original art, AI, UX, balancing and testing**, a small vertical slice is a multi-week effort; a polished interconnected sandbox is a substantially longer multi-month project. Actual timing depends heavily on art/animation detail and the amount of code reuse. Do not treat the roadmap as a promised delivery schedule.

16. Implementation order within the codebase

1. **State and saves first:** Serializable map changes, wallet, quest flags, schemas and migrations.
2. **Perfect the avatar:** Move, face, walk, dig, interact, attack, hit, damage, faint, respawn. Doors and building footprints accurate.
3. **Map ownership and terrain:** Diggable tiles, walls, water, ownership, path connectivity, dynamic collision.
4. **One complete farm loop:** Seed stock decreases, plot changes, watering, dawn growth, harvest, shipping, transaction.
5. **One complete building loop:** Pay cost, validate tile placement, construct, update service and save.
6. **One complete NPC loop:** Schedule, need, relationship change, journal record and shop access.
7. **One complete consequence chain:** Public pump choice changes faction standing, water supply and a later conversation.

8. **One complete danger chain:** Forecast storm, prepare drainage, trigger flood, apply damage, repair recovery.
9. **Expand content only after those loops pass QA.**

Automated tests to implement early

- Farming: crop advances **exactly once per day**; rain correctly waters; winter planting respects crop rules.
- Economy: purchases cannot overdraw wallets; shipping never pays twice; negative stock is impossible.
- Construction: blocked door footprints fail validation; paths remain reachable; demolished bridge removes access/coverage.
- Combat: one enemy hit applies damage once despite overlapping frames; invulnerability timers end; death/recovery has no item duplication.
- Reputation: unwitnessed theft does not trigger instant global punishment; restitution adjusts the correct faction and heat variables.
- Event director: identical seed/state leads to identical daily roll; mitigation lowers expected risk; opening 3 days cannot spawn catastrophic disasters.
- Saves: serialize → reload → continue has equivalent world state; import rejects invalid or incompatible records cleanly; schema migrations preserve content.
- Deployment: nested `/scurry/` asset URLs all resolve on GitHub Pages; browser test verifies no blank canvas / WebGL errors.

Playtest questions that actually matter

- After 10 minutes, does the player understand *why* the town changed?
- Is digging fun without farming or quest rewards?
- Do meaningful choices remain after choosing a selfish path?
- Does player construction change routes visually and mechanically?
- Can the farm make money without turning into endless watering chores?
- Are attacks legible, do hits feel fair, and can the player recover from defeat?
- Are there enough threats to make building safety useful without destroying cozy play?
- Does the town bulletin connect consequences to the player's own actions?

---

17. IP, licensing and production assets

All code, characters, map layouts, dialogue, music, sound and finished sprite sheets for Scurry should be **original or appropriately licensed**. Borrow **genre conventions** and research-level principles from the referenced games, not recognisable sprites, map designs, UI templates, names, music, creature designs or storylines. Record licenses/credits for every third-party asset, including fonts and sound effects. If publishing source publicly, select an explicit code license and separately state the license for artwork and audio; one code license does not automatically clarify permission to reuse all game art.

For starter art, first make a **small style bible** with the worm front/back/side silhouettes, grass and dirt tiles, three building doors, NPC sprites and a combat sheet. Consistent 16px-scale silhouettes matter more than producing hundreds of assets immediately. Use placeholder art with clearly labeled replacements until interactions, saves and content pipelines work.

---

18. Final recommended first playable release

Build 'Scurry: Day One' first. One original worm sprite, Clovercross square, one small farm, one underground tunnel, three NPCs, a weather cycle, four tools, two crops, two enemy types, a simple 5-heart health HUD, a shell wallet, one buildable footbridge and one meaningful choice about the water-pump component. The game should run on Windows in Chrome/Edge/Firefox through a GitHub Pages URL, and save progress locally.

A player should be able to:

1. Wake up in their burrow and learn to dig.
2. Plant a crop, collect materials and earn shells.
3. Fight a grub, lose health, and recover.
4. Meet townsfolk and make a decision they remember.
5. Build or repair something that changes the navigable town.
6. See the result in a Town Bulletin, sleep, reload, and find the consequences still present.

That six-step loop is the proof of the whole game. Everything else should enlarge it, not replace it.

Sources and further technical reading

Game systems / original game history

1. R. Philip Bouchard, principal designer of the 1985 version, history of *The Oregon Trail*: https://www.philipbouchard.com/oregon-trail/early-versions.html
2. *The Oregon Trail* 1985 courseware manual (archived transcription/scanned source): https://www.scribd.com/document/959830676/Oregon-Trail-Manual-1985
3. *Harvest Moon* (1996) mechanics and game description: https://www.mobygames.com/game/6585/harvest-moon/
4. *Harvest Moon* SNES gameplay walkthrough: https://gamefaqs.gamespot.com/snes/562623-harvest-moon/faqs/52336
5. Age of Empires official resource guide: https://www.ageofempires.com/learn-to-play/control-resources-aoe2/
6. *SimCity Classic* original manual: https://oldgamesdownload.com/wp-content/uploads/SimCity_Classic_Manual_DOS_EN.pdf
7. *SimCity* original engine system analysis: https://danielwymark.com/micropolis-systems-atlas/
8. *The Sims Deluxe Edition* original manual and needs system: https://www.manuallib.com/download/2023-10-18/The%20Sims%E2%84%A2%20Deluxe%20Edition%20Manual.pdf
9. *Pokémon Red and Blue* original Game Boy play progression: https://bulbapedia.bulbagarden.net/wiki/Walkthrough%3APok%C3%A9mon_Red_and_Blue

Web game development and deployment

1. Phaser current documentation: https://docs.phaser.io/
2. Phaser 4 release notes: https://github.com/phaserjs/phaser/releases
3. Vite GitHub Pages guide: https://vite.dev/guide/static-deploy.html
4. Vite public base path: https://vite.dev/guide/build.html#public-base-path